

CSEN 296B-2 | Week 3.1 Lab Submission

Refined Requirement Baseline for AstraNotes

Student Name: Alex Shirazi

Date: April 13, 2026

Project: AstraNotes

Chosen Technical Path: Python

1. Refined Requirement Baseline

The requirements below are a direct refinement of the Week 1.2 Requirement Set. Each requirement has been reviewed for ambiguity, weak assumptions, and missing edge cases identified during the Week 2.1 and Week 2.2 planning work. Changes are noted inline. Requirements that were already precise are carried forward unchanged.

1.1 Functional Requirements

ID	Refined Requirement	Change from v1
FR-01	The system shall allow a user to create a new text note with a non-empty title and a body (which may be empty). The note shall be assigned a UUID, created_at timestamp, and modified_at timestamp automatically at creation time.	Added: title must be non-empty (enforced by ValidationLayer). Body may be empty. Timestamps are system-assigned, not user-provided.
FR-02	The system shall allow a user to edit the title or body of an existing note. On save, the modified_at timestamp shall be updated to the current time. The created_at timestamp shall not change.	Added: explicit constraint that created_at is immutable. Clarified that modified_at is always updated on save, not only when content changes.
FR-03	The system shall allow a user to delete a note by its UUID. If the UUID does not exist in the collection, the system shall surface a clear error rather than silently succeed or crash.	Added: behavior for deleting a non-existent note. Original was silent on this edge case.
FR-04	The system shall allow a user to toggle a note's is_private flag. When is_private is True, the note body shall be encrypted by PrivacyService before being written to disk, and decrypted by PrivacyService on retrieval. Changing is_private from True to False shall re-save the note with a plaintext body.	Added: behavior for toggling privacy off (decryption + plaintext re-save). Original only described the encrypt-on-save direction.

FR-05	The system shall persist all notes to local file storage using a one-file-per-note JSON format. On startup, all valid note files shall be loaded. Files that are missing, corrupt, or unreadable shall be skipped with an error logged; remaining notes shall load normally.	Added: explicit handling for corrupt or missing files on startup. Original implied all-or-nothing loading.
FR-06	The system shall support note metadata: UUID (id), created_at (datetime, UTC), modified_at (datetime, UTC), and tags (list of strings, may be empty). Tags shall be stored as entered and are case-sensitive.	Added: UTC requirement for timestamps. Clarified tags are case-sensitive and may be an empty list.
FR-07	The system shall allow keyword search across note titles and bodies. Search shall be case-insensitive. Private notes that are successfully decrypted shall be included in search results. Notes that cannot be decrypted shall be excluded from results with an error logged.	Added: explicit behavior for private notes in search. Original did not address whether encrypted content is searched.
FR-08	The system shall display notes sorted by modified_at descending by default. Notes with identical modified_at values shall be ordered by created_at descending as a stable tiebreaker.	Added: stable tiebreaker for equal modified_at values. Original left sort order undefined for ties.

1.2 Non-Functional Requirements

ID	Refined Requirement	Change from v1
NFR-01	The system shall complete note listing and keyword search within 2 seconds for a local collection of up to 500 notes on a machine meeting minimum development hardware (defined as: 8 GB RAM, SSD storage, modern dual-core CPU).	Added: hardware baseline definition. Original stated the performance target without defining the measurement environment.
NFR-02	The system architecture shall enforce separation between the business logic layer (NoteService), the storage layer (NoteRepository / JsonFileRepository), and the privacy layer (PrivacyService) such that each can be unit-tested independently without instantiating the others.	Added: testability as an explicit constraint. Named the layers. Original was vague about what 'separation' required in practice.
NFR-03	The application shall load and display the note list within 3 seconds of launch on a machine meeting the hardware baseline defined in NFR-01, measured from process start to first interactive display.	Added: measurement definition (process start to first interactive display). Referenced NFR-01 hardware baseline for consistency.

1.3 Security, Privacy, Reliability, and Governance Requirements

ID	Refined Requirement	Change from v1
----	---------------------	----------------

SPR-01	Private notes (is_private == True) shall have their body encrypted using Fernet symmetric encryption (Python cryptography library) before any write to disk. The encryption key shall never be stored in plaintext in source code or in the notes data directory. Key management strategy shall be documented in the Architecture Decision Log before Sprint 1 begins.	Added: key storage constraint. Added requirement to document key management in ADL before Sprint 1. Original deferred key management without a deadline.
SPR-02	The system shall catch all storage-layer exceptions (FileNotFoundError, PermissionError, JSONDecodeError, and equivalent) and translate them into application-level error messages that do not expose file paths, stack traces, or internal state to the user or CLI output.	Added: specific exception types. Added constraint against exposing file paths. Original said 'non-technical error messages' without specifying what must be hidden.
SPR-03	All core components (NoteRepository, PrivacyService, ValidationLayer) shall have at least one automated unit test covering their primary behavior before each milestone delivery. Tests shall run without external dependencies (no real file system, no real encryption key required).	Added: tests must run without external dependencies (use mocking/tmp dirs). Original required tests but did not prevent tests that depend on real infrastructure.
SPR-04	Any third-party library added to the project shall be recorded in requirements.txt with a pinned version and a one-line comment explaining its purpose and confirming its license is permissive (MIT, Apache 2.0, BSD, or equivalent). Libraries without confirmed permissive licenses shall not be added.	Added: pinned versions. Added license confirmation requirement. Original said 'document purpose and license' without requiring pinning or license validation.

2. Ambiguity Review

The following ambiguities were identified in the original Week 1.2 requirement set by reviewing each requirement against the user stories and acceptance criteria from Week 2.2. Each item describes the ambiguity, why it matters, and how it was resolved.

A-01 — FR-01 — What counts as a valid title?

Ambiguity: The original requirement said 'a title and body' without specifying whether an empty string is acceptable. This matters because the ValidationLayer behavior and the US-01 acceptance criteria depend on having a clear rule.

Resolution: Resolved: title must be non-empty. Body may be empty. This is now enforced by ValidationLayer (raises ValidationError on empty title) and is explicit in the refined FR-01.

A-02 — FR-04 — What happens when privacy is toggled off?

Ambiguity: The original requirement described encrypting on save when is_private is True but said nothing about the reverse: what happens when a user marks a previously private note as non-private. This creates an undefined state where a note might have an encrypted body but is_private == False.

Resolution: Resolved: toggling is_private from True to False re-saves the note with a decrypted plaintext body. The note file on disk must always match the current privacy flag.

A-03 — FR-07 — Are private notes included in search?

Ambiguity: The original search requirement said 'search notes by title or body content' without addressing private notes. If private notes are encrypted on disk, searching their body requires decryption. The original was silent on whether search should attempt this.

Resolution: Resolved: private notes that decrypt successfully are included in search. Notes that cannot be decrypted are excluded and an error is logged. This is consistent with the layered architecture where PrivacyService is called before any content is processed.

A-04 — NFR-01 and NFR-03 — On what hardware?

Ambiguity: Both performance requirements stated time targets (2 seconds for search, 3 seconds for startup) without defining the measurement environment. A low-spec machine could fail these targets even on a well-implemented system.

Resolution: Resolved: a hardware baseline is defined in NFR-01 (8 GB RAM, SSD, modern dual-core CPU) and referenced in NFR-03. This makes the performance targets testable.

A-05 — SPR-01 — Where is the encryption key stored?

Ambiguity: The original requirement required Fernet encryption but said nothing about where the key lives. This is the most significant security gap in the original set. A key stored in the notes directory or hardcoded in source code would make encryption meaningless.

Resolution: Resolved: key must not be stored in source code or the notes directory. Key management strategy must be documented in the ADL before Sprint 1. The specific mechanism (environment variable, OS keychain, user prompt) is deferred but bounded.

A-06 — SPR-03 — What does 'unit test' require?

Ambiguity: The original requirement said 'automated unit test' without specifying whether tests could rely on real infrastructure (real files, real encryption keys). A test that writes to the real file system is technically an integration test and can fail in CI environments without that infrastructure.

Resolution: Resolved: unit tests must run without external dependencies. File system interactions should use temporary directories; encryption tests should use in-memory test keys.

3. Edge-Case Review

The following edge cases were identified by walking through each user story acceptance criteria and asking: what could go wrong that the requirement does not currently address? Cases are grouped by the component they affect.

Storage Layer (JsonFileRepository)

- **Note file is corrupt or malformed JSON on startup**

FR-05 now specifies that corrupt files are skipped with an error logged. The app must not crash or refuse to start.

- **Note data directory does not exist on first launch**

JsonFileRepository must create the data directory if it does not exist. Not addressed in original; added as an implementation constraint in sprint-zero-plan.md.

- **Two notes are saved in rapid succession with the same modified_at timestamp**

FR-08 now requires a stable tiebreaker (created_at descending). Without this, sort order is non-deterministic.

- **A note file exists on disk but its UUID in the filename does not match the UUID in the JSON body**

JsonFileRepository should use the UUID from the JSON body as the authoritative value and log a warning about the filename mismatch. Not addressed in original.

PrivacyService

- **Decryption fails because the key has changed since the note was encrypted**

PrivacyService must catch decryption failures and surface a clear error. The note should not be returned with garbled content. Addressed in refined FR-07 for search; applies to all retrieval operations.

- **A note is marked private but PrivacyService is unavailable or key is missing**

The system must not silently return unencrypted content. It must raise an error and block the read. This is an implicit constraint of SPR-01 that is now explicit.

- **Toggling privacy off when decryption fails**

If a private note cannot be decrypted, toggling `is_private` to `False` should not succeed silently. The system must inform the user that the note body could not be recovered.

ValidationLayer

- **Title contains only whitespace (e.g., spaces or tabs)**

A title of ' ' passes a naive non-empty check. ValidationLayer must strip whitespace before validation so that a whitespace-only title is treated as empty.

- **Tags list contains duplicate entries**

FR-06 does not prohibit duplicate tags. For now this is allowed (preserves user intent), but it should be noted as a known limitation in the planning docs.

- **Note body exceeds an unexpectedly large size**

No size limit is defined. For a local 500-note app this is unlikely to be a practical problem, but a soft warning at a reasonable threshold (e.g., 100 KB body) is a reasonable Sprint 2 consideration.

Search (FR-07)

- **Search is called on an empty note collection**

Must return an empty result set, not an error. The original acceptance criteria covered the no-match case but assumed at least one note exists.

- **Search keyword contains special regex characters**

If keyword matching is implemented using regex, characters like '.', '', '(' could cause unexpected behavior. The implementation must either escape the keyword or use a non-regex string search.*

4. How AI Helped and What I Refined

AI was used in three phases to produce this refined baseline: generating a first-pass ambiguity list, generating a first-pass edge-case list, and drafting refined requirement language.

What AI helped generate:

- A structured list of potential ambiguities in the original FR and NFR set. The AI correctly flagged the missing hardware baseline in NFR-01/NFR-03 and the undefined behavior for toggling `is_private` off (A-02). These matched my own review and were included.
- An initial edge-case list for the storage layer. Most of the `JsonFileRepository` edge cases (corrupt files, missing directory) were in the AI's first draft and were accurate given the architecture.
- Draft language for the refined requirements. The AI produced clean, specific wording for several requirements, particularly FR-05 and SPR-02.

What I changed or rejected:

- The AI flagged 'no offline support' as an ambiguity. AstraNotes is a local-only app by design; this is not an ambiguity, it is a scope decision already documented in the architecture. Rejected.
- The AI suggested adding a requirement for 'note versioning / undo history.' This is a new feature, not a refinement of an existing requirement. Rejected and parked as a future backlog idea.
- For SPR-01, the AI proposed requiring OS-level keychain integration as the key management solution. This is a reasonable approach but it is an implementation decision, not a requirement. Revised to require that the decision be documented in the ADL before Sprint 1, leaving the mechanism open.
- The AI's edge-case list included 'concurrent write from two users.' AstraNotes is a single-user local app; this is not a realistic edge case for this scope. Removed.

The refinement process confirmed that the Week 1.2 requirements were well-scoped but had several meaningful gaps — particularly around privacy edge cases and undefined failure modes. The six ambiguities and eleven edge cases identified above are the ones that would have caused real implementation confusion in Sprint 1 if left unaddressed.